Machine Learning – CSE 815

8th Semester B.Sc. Engg. Examination – **2020**

University of Chittagong, Dept. of CSE

New Topics Only (Repeated questions cross-referenced to 2021 solutions)

Note on Repeated Questions: Questions that appeared in both 2020 and 2021 exams are marked with a reference box (like this one) pointing to the 2021 solution document. Only *new* or *different* questions from the 2020 paper are fully answered here.

Contents

1 Q1(a): Neural Network – Layers, Weights, Activation Functions, Error 3

2 Q1(b): Artificial Neuron and Artificial Neural Network 3

3 Q1(c): ANN Training – Widrow-Hoff / LMS Rule 4

4 Q1(d): Supervised vs Unsupervised vs Hybrid ANN 5

5 Q2: Genetic Algorithm 6

6 Q3(b): CART 9

7 Q3(c): Boyer-Moore String Matching 10

8 Q3(d): Edit Distance 11

9 Q3(e): String Matching with Error and Don't-Care Symbol 11

10 Q4(a)(b): SVM – Support Vectors and Kernel Trick 12

11 Q4(c): K-Means Algorithm 12

12 Q4(d): K-Means with $k = 3$ (CGPA Data) 13

13 Q5(b): Neuron Output Calculation 14

14 Q5(c): Single-Layer Network Architecture 14

15 Q5(d): Multiple Layers of Neurons 15

16 Q5(e): Perceptron Rule **15**

17 Q6(a): Feature Space and Linear Separability **16**

18 Q6(b): Banana/Pineapple Perceptron **16**

19 Q7: Instance-Based Learning and KNN **17**

20 Q8: Bayesian Learning and Q-Learning **21**

Q1(a): Neural Network – Layers, Weights, Activation Functions, Error

Question: Define: (i) input/hidden/output layers, (ii) weights, (iii) activation functions, (iv) error.

⇒ **Same/Similar Question:** This question also appeared as **Q3(a) in the 2021 exam**. Refer to the 2021 solution document for the full answer.

For completeness, a brief summary:

- **Input Layer:** Receives raw input features $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$. No computation — just passes data forward.
- **Hidden Layer(s):** Intermediate layers between input and output. They extract features and learn non-linear patterns.
- **Output Layer:** Produces the final prediction $\hat{y}$.
- **Weights (W):** Numerical values on connections between neurons. They determine the strength of each connection:

$$n = \sum_i w_i x_i + b = W\mathbf{p} + b$$

- **Activation Functions:** Transform the net input n into an output a. Examples:

$$\text{Sigmoid: } a = \frac{1}{1+e^{-n}}, \quad \text{ReLU: } a = \max(0, n), \quad \text{Hardlim: } a = \begin{cases} 1 & n \geq 0 \\ 0 & n < 0 \end{cases}$$

- **Error:** Difference between desired output t and actual output a:

$$e = t - a$$

Q1(b): Artificial Neuron and Artificial Neural Network

Question: Write short notes on: (i) Artificial Neurons, (ii) Artificial Neural Network.

Answer:

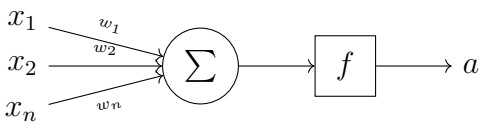
(i) Artificial Neuron (Perceptron):

An artificial neuron mimics a biological neuron. It:

1. Receives inputs $x_1, x_2, \ldots, x_n$

2. Multiplies each by a weight: $w_1x_1 + w_2x_2 + \cdots + w_nx_n$
3. Adds a bias b
4. Passes through an activation function f

$$a = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(W\mathbf{p} + b)$$



(ii) Artificial Neural Network (ANN):

An ANN is a collection of artificial neurons organized in *layers* (input, hidden, output), connected by weights. It is inspired by the human brain.

Key Properties:

- Learns from training data by adjusting weights
- Can approximate any continuous function (Universal Approximation Theorem)
- Used for: classification, regression, image recognition, NLP

Types: Feedforward, Recurrent (RNN), Convolutional (CNN), Hopfield, Hamming, etc.

Q1(c): ANN Training – Widrow-Hoff / LMS Rule

Question: Training set: $x^1 = (1, 0, 1)^T \to -1$, $x^2 = (0, -1, -1)^T \to 1$, $x^3 = (-1, -0.5, -1)^T \to 1$. Learning rate $\alpha = 0.1$, initial weights $w_0 = (1, -1, 0)^T$. Train the neural network.

Answer:

We use the **Widrow-Hoff (LMS / Delta) Rule**:

$$w_{\text{new}} = w_{\text{old}} + \alpha \cdot e \cdot x^T, \quad e = t - a, \quad a = W \cdot x$$

(Note: this is a linear neuron — no hard threshold, $a = W \cdot x$)

Initial: $W = [1,\ -1,\ 0]$

— **Iteration 1: Input** $x^1 = (1, 0, 1)^T$**, target** $t_1 = -1$ —

$$a_1 = W \cdot x^1 = (1)(1) + (-1)(0) + (0)(1) = 1$$

$$e_1 = t_1 - a_1 = -1 - 1 = -2$$

$$W \leftarrow [1, -1, 0] + 0.1 \times (-2) \times [1, 0, 1] = [1, -1, 0] + [-0.2, 0, -0.2] = [0.8,\ -1,\ -0.2]$$

— **Iteration 2: Input** $x^2 = (0, -1, -1)^T$**, target** $t_2 = 1$ —

$$a_2 = (0.8)(0) + (-1)(-1) + (-0.2)(-1) = 0 + 1 + 0.2 = 1.2$$

$$e_2 = 1 - 1.2 = -0.2$$

$$W \leftarrow [0.8, -1, -0.2] + 0.1 \times (-0.2) \times [0, -1, -1] = [0.8, -1, -0.2] + [0,\ 0.02,\ 0.02] = [0.8,\ -0.98,\ -0.18]$$

— **Iteration 3: Input** $x^3 = (-1, -0.5, -1)^T$**, target** $t_3 = 1$ —

$$a_3 = (0.8)(-1) + (-0.98)(-0.5) + (-0.18)(-1) = -0.8 + 0.49 + 0.18 = -0.13$$

$$e_3 = 1 - (-0.13) = 1.13$$

$$W \leftarrow [0.8, -0.98, -0.18] + 0.1 \times 1.13 \times [-1, -0.5, -1]$$

$$= [0.8, -0.98, -0.18] + [-0.113,\ -0.0565,\ -0.113] = [0.687,\ -1.037,\ -0.293]$$

After one full pass (Epoch 1): $W \approx [0.687,\ -1.037,\ -0.293]$

Training continues for more epochs until the mean squared error $E = \frac{1}{2}\sum e^2$ converges to a minimum.

Q1(d): Supervised vs Unsupervised vs Hybrid ANN

Question: Make a comparison among supervised, unsupervised and hybrid ANNs in terms of learning rule, architecture, learning algorithm and task.

Answer:

Aspect	Supervised ANN	Unsupervised ANN	Hybrid ANN
Learning Rule	Error-correction (Delta/LMS, Backpropagation)	Hebbian, Competitive learning	Combination of both
Architecture	Multilayer feed-forward; input-hidden-output layers	Single or two-layer; self-organizing	Multiple stages; some supervised, some unsupervised
Algorithm	Backpropagation, Perceptron rule, Widrow-Hoff	SOM (Kohonen), K-means, Hopfield	RBF networks, Neuro-fuzzy
Task	Classification, regression, prediction	Clustering, dimensionality reduction, pattern discovery	Classification with unsupervised feature learning
Requires Labels?	Yes	No	Partially
Examples	MLP, CNN for face recognition	SOM for data visualization	RBF network

Q2: Genetic Algorithm

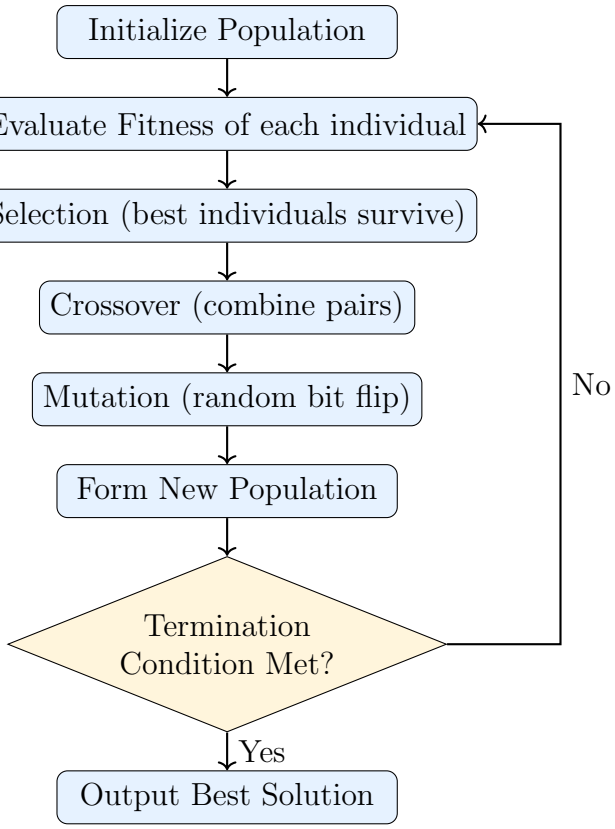
Q2(a): Genetic Algorithm Flowchart

Question: Draw a flow diagram to demonstrate the Genetic Algorithm process.

Answer:

Genetic Algorithm (GA) is a search/optimization algorithm inspired by natural evolution (selection, crossover, mutation).

Flow of GA:



Key Steps Explained:

- **Initialization:** Randomly generate an initial population of chromosomes (candidate solutions)
- **Fitness Evaluation:** Score each chromosome using the objective function $f(x)$
- **Selection:** Choose fitter individuals to reproduce (roulette wheel, tournament, etc.)
- **Crossover:** Exchange parts of two parent chromosomes to create offspring
- **Mutation:** Randomly change a gene to maintain diversity
- **Termination:** Stop when max generations reached, or fitness is good enough

Q2(b): Crossover Operations

Question: Chromosomes: x1=65413532, x2=87126601, x3=23921285, x4=41852094.
(i) One-point crossover at middle point between x1 and x2.
(ii) Two-point crossover at positions b and f between x3 and x4.

Answer:

Positions are labeled: **a b c d** — **e f g h** (1-indexed).

(i) One-Point Crossover between x1 and x2 at middle (after position 4):

	a	b	c	d		e	f	g	h
x1	6	5	4	1	|	3	5	3	2
x2	8	7	1	2	|	6	6	0	1
Offspring 1	6	5	4	1		6	6	0	1
Offspring 2	8	7	1	2		3	5	3	2

Result:

$$\text{Child 1} = 65416601, \quad \text{Child 2} = 87123532$$

(ii) Two-Point Crossover between x3 and x4 at positions b and f (after pos 2 and after pos 6):

a	|b	c	d	e	f|	g	h	
x3	2	3	9	2	1	2	8	5
x4	4	1	8	5	2	0	9	4
Offspring 1	2	1	8	5	2	0	8	5
Offspring 2	4	3	9	2	1	2	9	4

The **middle segment** (positions c–f) is swapped between the two parents.

Result:

$$\text{Child 1} = 21852085, \quad \text{Child 2} = 43921294$$

Q2(c): Genetic Algorithm for Function Optimization

Question: Let $f(x) = 2x - x^2/16$ on $[0, 31]$. Use a genetic algorithm to find the maximum. Relative fitness:

$$g(x) = \frac{f(x)}{\sum_{x \in p^{(k)}} f(x)}$$

Answer:

Step 1 – Encoding: $[0, 31]$ needs 5 bits ($2^5 = 32$). Each chromosome is a 5-bit binary string.

Step 2 – Initial Population (randomly chosen):

Chromosome	**Binary**	**Decimal** x	$f(x) = 2x - x^2/16$
c_1	01101	13	$2(13) - 169/16 = 26 - 10.56 = 15.44$
c_2	11000	24	$2(24) - 576/16 = 48 - 36 = 12$
c_3	01000	8	$2(8) - 64/16 = 16 - 4 = 12$
c_4	10011	19	$2(19) - 361/16 = 38 - 22.56 = 15.44$
Total			**54.88**

Note: The theoretical maximum of $f(x) = 2x - x^2/16$ is found by:

$$f'(x) = 2 - \frac{x}{8} = 0 \implies x^* = 16, \quad f(16) = 32 - 16 = 16$$

So the maximum is $f(16) = 16$ at $x = 16$ (binary: `10000`).

Step 3 – Relative Fitness:

$$g(c_1) = \frac{15.44}{54.88} \approx 0.281, \quad g(c_2) = \frac{12}{54.88} \approx 0.219$$

$$g(c_3) = \frac{12}{54.88} \approx 0.219, \quad g(c_4) = \frac{15.44}{54.88} \approx 0.281$$

Step 4 – Selection: Higher fitness ⇒ higher probability of selection. c_1 and c_4 have the highest fitness.

Step 5 – Crossover (one-point, e.g. after bit 3):

$c_1 = 01101$, $c_4 = 10011$:

$$\text{Child } 1 = 011\mathbf{11} = 01111 \to x = 15, \; f(15) = 30 - 14.06 = 15.94$$

$$\text{Child } 2 = 100\mathbf{01} = 10001 \to x = 17, \; f(17) = 34 - 18.06 = 15.94$$

Both children are close to the optimum $x = 16$! The GA converges toward $x = 16$ over generations.

Q3(b): CART

Question: What is CART? Describe four important string problems in pattern recognition.

Answer:

CART (Classification and Regression Trees):

CART is a decision tree algorithm that builds *binary trees* (each node splits into exactly 2 branches). It can handle both classification and regression tasks.

Key features of CART:

- Uses **Gini Impurity** for classification splits:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2$$

- Uses **MSE** for regression splits
- Always produces **binary splits** (unlike ID3/C4.5 which can split into many branches)
- Supports **pruning** to avoid overfitting

⇒ **Same/Similar Question:** Decision Tree short notes appeared as **Q6(a) in the 2021 exam** and **Q3(a) in the 2020 exam**. Refer to the 2021 solution for the general Decision Tree answer. CART is a specific variant described above.

Four Important String Problems in Pattern Recognition:

1. **Exact String Matching:** Find all occurrences of a pattern P in text T. Algorithms: Naive, KMP, Boyer-Moore.
2. **Approximate String Matching (String Matching with Error):** Find pattern occurrences allowing up to k mismatches or edit operations. Uses Edit Distance. Example: matching "colour" with "color".
3. **String Matching with Don't-Care Symbols:** Pattern contains wildcard characters (e.g., '?') that match any character. Example: Pattern "a?c" matches "abc", "acc", "axc", etc.
4. **Regular Expression Matching:** Pattern is a regular expression (e.g., `a*b+`). Uses finite automata (NFA/DFA). Used in compilers, text search tools.

Q3(c): Boyer-Moore String Matching

Question: Write down Boyer-Moore String Matching algorithm and explain with an example.

Answer:

Boyer-Moore Algorithm is one of the most efficient string matching algorithms. It searches from *right to left* within the pattern and uses two heuristics to skip comparisons:

Two Heuristics:

- **Bad Character Heuristic:** When a mismatch occurs at character c in text, shift pattern so the rightmost occurrence of c in the pattern aligns with the mismatch position. If c doesn't appear in pattern, shift past it entirely.
- **Good Suffix Heuristic:** When a suffix of the pattern matches, shift pattern to next occurrence of that suffix, or to the longest prefix that matches a suffix.

The shift used is max(bad character shift, good suffix shift).

Example: Text T = "ABCBABCABCABC", Pattern P = "ABCABC"

Bad Character Table for P = A B C A B C:

A	B	C	(others)
3	4	5	−1

(Rightmost occurrence index of each character in pattern)

Matching Process:

Alignment 1: Align P at position 0. Compare right to left. $P[5]$=C matches $T[5]$=C; $P[4]$=B matches $T[4]$=B; ... match found partially, calculate shift.

Boyer-Moore can skip large portions of text, making it very fast in practice: $O(n/m)$ best case, $O(nm)$ worst case.

Q3(d): Edit Distance

⇒ **Same/Similar Question:** Edit Distance appeared as **Q6(b) in the 2021 exam** with the same "exercised" vs "executed" example. Refer to the 2021 solution document for the full DP table and answer (**Edit Distance = 3**).

Q3(e): String Matching with Error and Don't-Care Symbol

Question: Write short notes on: (i) String matching with Error, (ii) String matching with the "Don't-Care" Symbol.

Answer:

(i) String Matching with Error (Approximate Matching):

Allows up to k errors (mismatches, insertions, deletions) when matching pattern P in text T.

Approach: Use dynamic programming similar to Edit Distance. Define $D[i][j]$ = minimum errors to match first i characters of P with a substring of T ending at position j.

$$D[i][j] = \min \begin{cases} D[i-1][j-1] + [P[i] \neq T[j]] & \text{(substitute or match)} \\ D[i-1][j] + 1 & \text{(deletion from pattern)} \\ D[i][j-1] + 1 & \text{(insertion into pattern)} \end{cases}$$

Example: P = "CAT", T = "CATS", $k = 1$. The algorithm finds "CAT" in T with 0 errors. "COT" would match with 1 error.

Applications: Spell checking, DNA sequence alignment, plagiarism detection.

(ii) String Matching with Don't-Care Symbol:

Pattern P contains a wildcard character (usually '?' or '*') that matches *any single character*.

Matching Rule:

$$P[i] \text{ matches } T[j] \iff P[i] = T[j] \text{ OR } P[i] = \texttt{?}$$

Example:

Pattern: `a?c?e` Text: `abcde`

P:	a	?	c	?	e
T:	a	b	c	d	e
Match:	✓	✓	✓	✓	✓

Result: **Match!** (The '?' matches any character.)

Algorithm: Standard string matching with modified comparison function. Time complexity: $O(nm)$.

Q4(a)(b): SVM – Support Vectors and Kernel Trick

⇒ **Same/Similar Question:** SVM questions Q4(a) and Q4(b) of the 2020 exam are **identical** to **Q6(c) and Q6(d) of the 2021 exam**. Refer to the 2021 solution document for complete answers on support vectors, maximum margin, kernel trick, and computing $M = \frac{2}{\|W\|}$.

Q4(c): K-Means Algorithm

Question: Write the algorithm for K-Means clustering.

Answer:

K-Means Algorithm:

Input: Dataset $X = \{x_1, x_2, \ldots, x_n\}$, number of clusters K
Output: K clusters $C_1, C_2, \ldots, C_K$ with centers $\mu_1, \mu_2, \ldots, \mu_K$

1. **Initialize:** Randomly select K data points as initial cluster centers $\mu_1, \mu_2, \ldots, \mu_K$
2. **Assignment Step:** For each data point x_i, assign it to the nearest cluster:
$$C_j = \{x_i : \|x_i - \mu_j\|^2 \leq \|x_i - \mu_k\|^2 \text{ for all } k\}$$
3. **Update Step:** Recompute each cluster center as the mean of its members:
$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$
4. **Convergence Check:** If cluster assignments did not change, **STOP**. Otherwise, go to Step 2.

Time Complexity: $O(n \cdot K \cdot I \cdot d)$, where n = points, K = clusters, I = iterations, d = dimensions.

Limitations:

- Sensitive to initial center placement
- Assumes spherical clusters of similar size
- Must specify K in advance

Q4(d): K-Means with $k = 3$ (CGPA Data)

Question: Cluster students A=3.45, B=3.78, C=2.98, D=3.24, E=4.0, F=3.9 into $k = 3$ clusters. Initial centers: A, C, E. Use Euclidean distance.

Answer:

Initial Centers: $\mu_1 = A = 3.45$, $\mu_2 = C = 2.98$, $\mu_3 = E = 4.0$

Iteration 1 – Assign to nearest center:

Student	CGPA	$\lvert x-\mu_1\rvert$	$\lvert x-\mu_2\rvert$	$\lvert x-\mu_3\rvert$	Cluster
A	3.45	0.00	0.47	0.55	C_1
B	3.78	0.33	0.80	0.22	C_3
C	2.98	0.47	0.00	1.02	C_2
D	3.24	0.21	0.26	0.76	C_1
E	4.00	0.55	1.02	0.00	C_3
F	3.90	0.45	0.92	0.10	C_3

Update Centers:

$$\mu_1 = \frac{3.45 + 3.24}{2} = 3.345, \quad \mu_2 = \frac{2.98}{1} = 2.98, \quad \mu_3 = \frac{3.78 + 4.0 + 3.9}{3} = 3.893$$

Iteration 2 – Reassign with new centers:

Student	CGPA	$\lvert x-3.345\rvert$	$\lvert x-2.98\rvert$	$\lvert x-3.893\rvert$	Cluster
A	3.45	0.105	0.47	0.443	C_1
B	3.78	0.435	0.80	0.113	C_3
C	2.98	0.365	0.00	0.913	C_2
D	3.24	0.105	0.26	0.653	C_1
E	4.00	0.655	1.02	0.107	C_3
F	3.90	0.555	0.92	0.007	C_3

Same assignments as Iteration 1 $\Rightarrow$ **Converged!**

Final Clusters:

- **Cluster 1 (Mid-range):** A (3.45), D (3.24) – Center: 3.345
- **Cluster 2 (Lowest):** C (2.98) – Center: 2.98
- **Cluster 3 (Highest):** B (3.78), E (4.0), F (3.9) – Center: 3.893

Q5(b): Neuron Output Calculation

Question: Two-input neuron: $b = 1.2$, $W = [3\ 2]$, $\mathbf{p} = [-5\ 6]^T$. Calculate output for: (i) symmetrical hard limit, (ii) saturating linear transfer function.

Answer:

Net Input:

$$n = W \cdot \mathbf{p} + b = [3\ 2] \begin{bmatrix} -5 \\ 6 \end{bmatrix} + 1.2 = (3)(-5) + (2)(6) + 1.2 = -15 + 12 + 1.2 = -1.8$$

(i) Symmetrical Hard Limit (Sign function):

$$a = \text{satlins}(n) = \begin{cases} +1 & n \geq 0 \\ -1 & n < 0 \end{cases}$$

$$n = -1.8 < 0 \implies \boxed{a = -1}$$

(ii) Saturating Linear Transfer Function:

$$a = \text{satlin}(n) = \begin{cases} 0 & n < 0 \\ n & 0 \leq n \leq 1 \\ 1 & n > 1 \end{cases}$$

$$n = -1.8 < 0 \implies \boxed{a = 0}$$

Q5(c): Single-Layer Network Architecture

Question: A single-layer neural network has 6 inputs and 2 outputs. Outputs are continuous over $[0, 1]$. (i) How many neurons are required? (ii) What are the dimensions of the weight matrix?

Answer:

(i) Number of Neurons:

In a single-layer network, the output layer IS the only layer. Since there are **2 outputs**, we need **2 neurons**.

(ii) Weight Matrix Dimensions:

Each neuron is connected to all 6 inputs. The weight matrix W has:

$$W \in \mathbb{R}^{S \times R} = \mathbb{R}^{2 \times 6}$$

where $S = 2$ (neurons/outputs), $R = 6$ (inputs).

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} & w_{1,4} & w_{1,5} & w_{1,6} \\ w_{2,1} & w_{2,2} & w_{2,3} & w_{2,4} & w_{2,5} & w_{2,6} \end{bmatrix}$$

Bias vector: $\mathbf{b} \in \mathbb{R}^{2\times 1}$.

Since outputs are continuous over $[0, 1]$, the activation function is the **Sigmoid** $\sigma(n) = \frac{1}{1+e^{-n}}$ or **Saturating Linear**.

Q5(d): Multiple Layers of Neurons

Question: Describe multiple layers of neurons with necessary figures.

Answer:

A **Multi-Layer Neural Network** (MLNN) has:

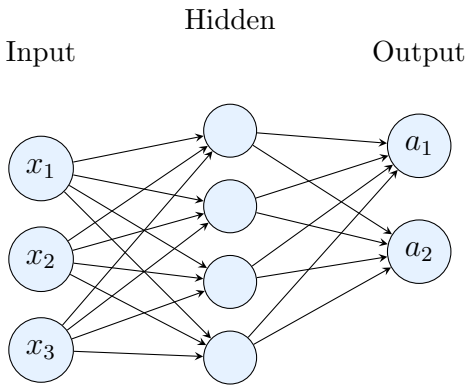
- **Layer 1 (Input Layer):** Receives input features. No computation.
- **Layer 2, 3, . . . (Hidden Layers):** Extract increasingly abstract features.
- **Last Layer (Output Layer):** Produces final output.

Mathematical representation:

For a 3-layer network:

$$\mathbf{a}^1 = f^1(W^1\mathbf{p} + \mathbf{b}^1) \quad \text{(hidden layer)}$$

$$\mathbf{a}^2 = f^2(W^2\mathbf{a}^1 + \mathbf{b}^2) \quad \text{(output layer)}$$



Advantages of Multiple Layers:

- Can learn *non-linear* decision boundaries
- Each layer extracts higher-level features
- Universal approximation: can approximate any function

Q5(e): Perceptron Rule

⇒ **Same/Similar Question:** This is **identical** to **Q7(b) in the 2021 exam** (same input vectors, targets, and initial weights). Refer to the 2021 solution document for the complete step-by-step solution.
Answer: Final $W = [-2 \ \ -3]$, $b = 1$.

Q6(a): Feature Space and Linear Separability

Question: What do you understand by feature space? What is linearly separable and not separable? Explain with example.

⇒ **Same/Similar Question:** This question also appeared as **Q5(b) in the 2021 exam**. Refer to the 2021 solution document for the full answer.

Answer: (Brief summary)

Feature Space: An n-dimensional space where each dimension represents one feature. Each data point is a vector in this space.

Linearly Separable: Two classes can be separated by a single straight line (2D), plane (3D), or hyperplane (nD).

Not Linearly Separable: No single hyperplane can separate the classes (e.g., XOR problem). Requires a *kernel* (SVM) or *hidden layers* (ANN) to solve.

Q6(b): Banana/Pineapple Perceptron

Question: Fruit sensor: $p = [\text{shape, texture, weight}]^T$. Two fruits: Banana ($p_1$) and Pineapple ($p_2$). Design a perceptron; test with different inputs.

Answer:

Prototype Templates:

A banana is: elongated (shape $= -1$), smooth (texture $= +1$), light (weight $= -1$):

$$p_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} \text{ (Banana)}$$

A pineapple is: round (shape $= +1$), rough (texture $= -1$), heavy (weight $= +1$):

$$p_2 = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \text{ (Pineapple)}$$

Perceptron Design using Hebb Rule:

Targets: $t_1 = +1$ (Banana), $t_2 = -1$ (Pineapple)

$$W = t_1 p_1^T + t_2 p_2^T = (+1)[-1\ 1\ -1] + (-1)[1\ -1\ 1]$$
$$W = [-1\ 1\ -1] + [-1\ 1\ -1] = [-2\ 2\ -2]$$
$$b = t_1 + t_2 = 1 + (-1) = 0$$

Testing:

Test p_1 (Banana):

$$n = [-2\ 2\ -2] \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} + 0 = 2 + 2 + 2 = 6 > 0 \implies a = +1 \checkmark \text{ Banana}$$

Test p_2 (Pineapple):

$$n = [-2\ 2\ -2] \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} + 0 = -2 - 2 - 2 = -6 < 0 \implies a = -1 \checkmark \text{ Pineapple}$$

Test a noisy input $p_? = [1, 1, -1]^T$ (round, smooth, light):

$$n = [-2\ 2\ -2] \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} = -2 + 2 + 2 = 2 > 0 \implies a = +1 \rightarrow \text{Banana}$$

⇒ **Same/Similar Question:** Hamming and Hopfield network design is similar to **Q8(c)(d) in the 2021 exam** (with Orange/Apple). The same methodology applies here with Banana/Pineapple prototypes. Refer to the 2021 solution for the full Hamming and Hopfield design methodology.

Q7: Instance-Based Learning and KNN

Q7(a): Instance-Based Learning

Question: What do you understand about Instance-Based Learning?

Answer:

Instance-Based Learning (IBL) (also called *lazy learning* or *memory-based learning*) is a class of ML algorithms that:

- **Store** all training examples instead of building an explicit model
- **Delay** all processing until a query arrives (hence "lazy")
- **Classify** new instances by comparing to stored examples

Key Properties:

- No training phase (just memorization)
- Classification is expensive (compare to all stored examples)
- Can adapt to local patterns
- Sensitive to irrelevant features and noisy data

Examples: K-Nearest Neighbor (KNN), Case-Based Reasoning, BNGE.

Q7(b): K-Nearest Neighbor (KNN)

Question: Describe the k-nearest neighbor learning algorithm.

Answer:

KNN Algorithm:

Training Phase: Simply store all training examples $\{(x_i, y_i)\}_{i=1}^{n}$.

Classification of new point x_q:

1. Compute distance from x_q to all stored examples using a distance metric $d(x_q, x_i)$
2. Select the k nearest neighbors $N_k(x_q)$
3. Assign class by **majority vote**:

$$\hat{y} = \arg\max_{c} \sum_{x_i \in N_k(x_q)} \mathbf{1}[y_i = c]$$

Effect of k:

- **Small k (e.g., $k = 1$):** Very sensitive to noise, low bias, high variance
- **Large k:** Smoother boundary, high bias, low variance
- Common choice: k = odd number to avoid ties; use cross-validation to select best k

Time Complexity: $O(n \cdot d)$ per query, where n = training points, d = dimensions.

Q7(c): Distance Metrics

Question: What is a distance metric? What properties must it satisfy? Define any three: Minkowski, Euclidean, Manhattan, Hamming, Maximum norm.

Answer:

A **distance metric** $d(x, y)$ must satisfy four properties:

Property	Condition	Meaning
Non-negativity	$d(x, y) \geq 0$	Distance is always ≥ 0
Identity	$d(x, y) = 0 \iff x = y$	Zero distance means same point
Symmetry	$d(x, y) = d(y, x)$	Distance is direction-independent
Triangle inequality	$d(x, z) \leq d(x, y) + d(y, z)$	Direct route is shortest

(i) Minkowski Norm (L_p norm):

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

Generalization: $p = 1$ gives Manhattan, $p = 2$ gives Euclidean, $p \to \infty$ gives Maximum norm.

(ii) Euclidean Norm (L_2, $p = 2$):

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Most common; represents straight-line distance. Example: $x = (1, 2)$, $y = (4, 6)$: $d = \sqrt{9 + 16} = 5$.

(iii) Manhattan Norm (L_1, $p = 1$):

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Also called "city block" distance. Example: $x = (1, 2)$, $y = (4, 6)$: $d = 3 + 4 = 7$.

(iv) Hamming Distance:

Number of positions where two binary strings differ:

$$d_H(x, y) = \sum_{i=1}^{n} \mathbf{1}[x_i \neq y_i]$$

Example: $x = 10110$, $y = 11010$: positions 2 and 3 differ, $d_H = 2$.

(v) Maximum (Chebyshev) Norm (L_∞, $p \to \infty$):

$$d(x, y) = \max_i |x_i - y_i|$$

Example: $x = (1, 2)$, $y = (4, 6)$: $d = \max(3, 4) = 4$.

Q7(d): Sequential Covering Algorithm

Question: Write down the Sequential Covering Algorithm.

Answer:

Sequential Covering Algorithm (also called *Separate and Conquer*) is a rule induction method that learns a set of IF-THEN rules to classify data.

Algorithm:

1. $\mathcal{R} \leftarrow \emptyset$ (empty rule set)
2. **While** positive examples remain in training set:
   (a) Learn a rule r that:
   - Covers as many *positive* examples as possible
   - Covers as few *negative* examples as possible

   (Use a greedy search: add conditions that maximize accuracy)
   (b) Add r to $\mathcal{R}$
   (c) Remove all examples covered by r from the training set
3. Add a default rule (majority class) to $\mathcal{R}$
4. **Return** $\mathcal{R}$

Example:

Training data has examples of "PlayTennis = Yes/No".

Rule learned: IF Outlook=Sunny AND Humidity=Normal THEN PlayTennis=Yes

All examples covered by this rule are removed; the process repeats for remaining positive examples.

Algorithms based on this: RIPPER, CN2, AQ.

Q7(e): Inductive vs Analytical Learning

Question: What is the difference between inductive and analytical learning methods?

Answer:

Aspect	Inductive Learning	Analytical Learning
Definition	Learns from examples/data	Learns from prior knowledge/logic
Input	Training examples (x, y)	Background knowledge + examples
Method	Generalize from observed data	Use deduction and logic
Examples	KNN, Decision Trees, ANN	Explanation-Based Learning (EBL)
Requires	Many examples	Strong domain knowledge
Output	Statistical model	Logical rules
Bias	Inductive bias (Occam's razor)	Deductive reasoning

Key Difference: Inductive learning *generalizes* from specific examples (bottom-up). Analytical learning *deduces* from general principles (top-down).

Q8: Bayesian Learning and Q-Learning

Q8(a): Machine Learning and Designing a Learning System

Question: What is Machine Learning? Describe the process of designing a learning system.

⇒ **Same/Similar Question:** The definition of Machine Learning appeared in **Q1(a) of the 2021 exam**. Refer there for the ML definition.

Process of Designing a Learning System:

1. **Define the Task:** What should the system learn? (Classification, regression, clustering, etc.)
2. **Collect Training Data:** Gather representative labeled/unlabeled examples.
3. **Choose Features:** Select the most informative attributes (feature engineering/selection).
4. **Choose the Learning Algorithm:** Based on task type, data size, and interpretability needs.
5. **Train the Model:** Fit the model to training data; minimize a loss function:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, \hat{y}_i)$$

6. **Evaluate:** Use validation/test data. Measure accuracy, F1, AUC, etc.
7. **Tune:** Adjust hyperparameters, regularization, architecture.
8. **Deploy:** Put model into production; monitor and retrain as needed.

Q8(b): Bayesian Classifiers

Question: Bayesian classifiers are statistical classifiers. Justify your answer.

Answer:

Bayesian classifiers are statistical because they are based on **Bayes' Theorem** – a fundamental result of probability theory:

$$P(C \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid C) \cdot P(C)}{P(\mathbf{x})}$$

where:

- $P(C \mid \mathbf{x})$ = posterior probability (class given features)

- $P(\mathbf{x} \mid C)$ = likelihood (features given class)
- $P(C)$ = prior probability of class
- $P(\mathbf{x})$ = evidence (normalizing constant)

Classification Decision:

$$\hat{C} = \arg\max_C P(C \mid \mathbf{x}) = \arg\max_C P(\mathbf{x} \mid C) \cdot P(C)$$

Why statistical?

- All decisions are based on probability distributions estimated from data
- Handles uncertainty naturally
- Makes optimal decisions when the true distribution is known
- Minimizes the probability of misclassification (Bayes optimal classifier)

Q8(c): Naive Bayes as Class Conditional Independence

Question: Why is Naive Bayesian Classification known as class conditional independence?

Answer:

Naive Bayes is called **class conditional independence** because it assumes that each feature x_i is *independent of every other feature, given the class label* C:

$$P(x_1, x_2, \ldots, x_n \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

This is the **naive** assumption (often not true in reality, but works well in practice).

Why it simplifies computation:

Without independence:

$$P(\mathbf{x} \mid C) = P(x_1 \mid C) \cdot P(x_2 \mid x_1, C) \cdot P(x_3 \mid x_1, x_2, C) \cdots$$

This requires exponentially many parameters.

With naive (conditional) independence:

$$P(\mathbf{x} \mid C) = P(x_1 \mid C) \cdot P(x_2 \mid C) \cdot P(x_3 \mid C) \cdots = \prod_{i=1}^{n} P(x_i \mid C)$$

Only $n \times K$ parameters needed (where K = number of classes).

Full Naive Bayes classifier:

$$\hat{C} = \arg\max_C P(C) \prod_{i=1}^{n} P(x_i \mid C)$$

Q8(d): Q-Learning

Question: What is Q learning? Describe the Q learning algorithm.

Answer:

Q-Learning is a **Reinforcement Learning** algorithm where an agent learns to take optimal actions in an environment to maximize cumulative reward, *without needing a model of the environment.*

Key Concepts:

- **State** s: Current situation of the agent
- **Action** a: What the agent can do
- **Reward** r: Feedback from environment
- **Q-value** $Q(s, a)$: Expected total future reward for taking action a in state s

Q-Value Update Rule (Bellman Equation):

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

where:

- α = learning rate $(0 < \alpha \leq 1)$
- γ = discount factor $(0 \leq \gamma < 1)$ — how much to value future rewards
- s' = next state after taking action a in state s
- r = immediate reward received

Q-Learning Algorithm:

1. Initialize $Q(s, a) = 0$ for all states s and actions a
2. **For each episode:**
   (a) Start in initial state s
   (b) **Repeat until terminal state:**
      i. Choose action a using policy (e.g., ε-greedy):

$$a = \begin{cases} \text{random action} & \text{with probability } \varepsilon \\ \arg\max_a Q(s,a) & \text{with probability } 1-\varepsilon \end{cases}$$

      ii. Take action a, observe reward r and next state s'
      iii. Update Q-table:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

iv. $s \leftarrow s'$

3. **Return** learned Q-table (policy: $\pi(s) = \arg\max_a Q(s, a)$)

Applications: Game playing (AlphaGo), robot navigation, resource management.

Advantage: Model-free (no need to know transition probabilities). Guaranteed to converge to optimal policy under certain conditions.